

Rover Coding Project

Rover.com was destroyed in a terrible Amazon Web Services and GitHub accident!
Thankfully, no dogs were harmed, but we will have to rebuild our site using data we retrieved from the Google search index.

Your task is to recreate a search ranking algorithm and compute search scores for our sitters.

Requirements

To recreate our search rankings, you'll create *a simple command-line program* that will read in and write out csv data. We ask that you also include tests that demonstrate your code is working as expected.

Additionally, there is a short *discussion question* to answer (see below) about how you would build this system in the real world.

Lastly, please include a README with instructions on how to setup and run your project locally, as well as a copy of the output CSV you generate. Note that we are primarily Mac users.

Please use the language that you feel will best show your skills.
Keep in mind that if you are brought for an in-person interview, you may be asked to continue building upon this solution. Feel free to use whatever libraries or packages you wish.

Finally, if you have any questions, don't hesitate to ask.

Recreating the Search Ranking Algorithm

We were able to write a script and scrape the Google index for all of the reviews customers have left for their stays with sitters.

We have saved that information in the attached CSV.

Your command-line program should import the data to hold in memory (no need to use an actual database) and use it to recreate our search algorithm. At Rover, we write domain-driven code, so using an Object Oriented or other domain-centric approach will set you up for success in the in person interview. For example, one viable approach is modeling your data as if you were going to save it using a relational database.

If you are using a language that doesn't traditionally use an OO or domain-centric approach, please discuss this with the person who sent you this prompt in advance of beginning work, in order to have the best chance of success if you advance to the in person interview.

Here's how the search ranking algorithm will work:

- For each sitter, we first calculate a Profile Score and a Ratings Score.

These are then used to calculate the overall Search Score, which is used for search rankings.

- The Profile Score is 5 times the fraction of the English alphabet comprised by the

distinct letters in what we've recovered of the sitter's name. For example, the sitter name `Leilani R.` has 6 distinct letters.

- The Ratings Score is the average of their stay ratings.

- The Search Score is a weighted average of the Profile Score and Ratings Score. When a sitter has no stays, their Search Score is equal to the Profile Score. When a sitter has 10 or more

stays, their Search Score is equal to the Ratings Score. The idea is that as a sitter gets more reviews, we will weigh the Ratings Score more heavily.

- Scores should contain exactly two decimal places.

Output a list of Sitters

Your program should output a csv called `sitters.csv`, containing the following columns:

- * Sitter email (`email`)
- * Sitter name (`name`)
- * Profile Score (`profile\_score`)
- * Ratings Score (`ratings\_score`)
- * Search Score (`search\_score`)

The csv should be sorted by Search Score (descending), sorting alphabetically on the sitter name as a tie-breaker.

Hint for Testing the Search Ranking Algorithm

Suppose there is a sitter whose Profile Score is 2.5 and who gets a rating of 5.0 with every stay. Then their Search Score should behave like this:

Stay	Search Score
0	2.50
1	2.75
2	3.00
3	3.25
4	3.50
5	3.75
6	4.00
7	4.25

```
| 8 | 4.50
| 9 | 4.75
| 10 | 5.00
| 11 | 5.00
| 12 | 5.00
```

Discussion Question

Imagine you are designing a Rover-like production web application based on the exercise you've just completed. The application will compute the search scores

for sitters, return a list of search results based on those scores, and display them to the user through a web UI. Please answer ****ONE**** of the following discussion questions about the approach you'd take:

- How would you adjust the calculation and storage of search scores in a production application?
- Describe a technical implementation for the frontend you would use to display a list of sitters and their scores. How would the frontend manage state as users interact with a page?
- What infrastructure choices might you make to build and host this project at scale? Suppose your web application must return fast search results with a peak of 10 searches per second.
- Describe how you would approach API design for a backend service to provide sitter and rank data to a client/web frontend.

Write your answer in the README inside your project github repo. Please keep your answer concise, but provide as much detail as you feel is necessary; we're not looking for a thesis, just an understanding of how you think about solutions and how you convey your thoughts in the written word.

When you're done with the project...

When you're done with the project, compress your project directory into a Zip file or similar, making sure to include the output file, `sitters.csv`. Then, reply to the email you received from us with your attachment.

Evaluation

Checklist:

- [] Have you modeled the data ingested from the CSV in a way that would support storage in a relational database?
- [] Are Profile, Rating, and Search Scores computed correctly?
- [] Does the output file include all necessary columns, and is it in descending order based on Search Score?
- [] Does the README include setup/running instructions (ideally for Mac)?
- [] Does the README include your answer to the Discussion Question?
- [] Have you included your CLI code, README, ****and output file**** in the Zip folder?
- [] Have you written tests to verify that your code is working as expected?

****Lastly:**** The work you create here should be representative of code that we'd expect to receive from you if you were hired tomorrow (proper abstractions, tests for the scoring algorithm calculation, best practices, etc).